

Foundational Implementations of Physics-Informed and Spectrally-Informed Neural Networks: A Prerequisite Study

Abstract

This report documents a sequence of foundational implementations carried out as preparatory work for research on fast Gauss–Newton solvers for spectral Physics-Informed Neural Networks (PINNs). Four self-contained notebooks were built, each isolating a single conceptual or computational challenge: (i) a vanilla PINN for the 1D viscous Burgers equation, establishing the standard residual–initial–boundary loss decomposition and verifying that automatic differentiation of nested derivatives is practical; (ii) a comparative study of collocation strategies (uniform, random, Chebyshev, Chebyshev domain-decomposed) and activations ($\tanh$, $\sin$) on the 1D Poisson equation, identifying $\sin$ activation with Chebyshev domain-decomposed collocation as the most accurate combination (L^2 error $\approx 2.1 \times 10^{-5}$ at $N = 320$); and (iii)–(iv) two Spectrally-Informed Neural Network (SINN) implementations for the fourth- and sixth-order linear evolution equations $u_t + u_{xxxx} = 0$ and $u_t + u_{xxxxx} = 0$, recasting each PDE as a system of decoupled ordinary differential equations in Fourier space. Both first-order PINN and SINN implementations on these high-order PDEs fail to converge in the Adam-only regime, exposing the stiffness and conditioning pathologies that motivate the use of second-order optimization and spectral compression in subsequent research.

Contents

1	Introduction and Motivation	2
2	Mathematical Background	3
2.1	The PINN Loss	3
2.2	Spectrally-Informed Neural Networks	3
3	Notebook 1: PINN for the 1D Viscous Burgers Equation	3
3.1	Problem Statement	3
3.2	Network and Loss	4
3.3	Collocation Points	4
3.4	Results	4
3.5	Takeaways	5
4	Notebook 2: Comparative Study on the 1D Poisson Equation	6
4.1	Problem and Goals	6
4.2	Chebyshev Nodes: Why Bother?	6
4.3	Four Collocation Strategies	6
4.4	Network and Training	7
4.5	Results	7
4.6	L-BFGS Refinement	8
4.7	Takeaways	9

5	Notebook 3: SINN for the 4th-Order Equation $u_t + u_{xxxx} = 0$	9
5.1	Problem and Manufactured Solution	9
5.2	PINN Baseline	9
5.3	SINN Reformulation	9
5.4	Results — An Honest Account	10
6	Notebook 4: SINN for the 6th-Order Equation $u_t + u_{xxxxx} = 0$	11
6.1	Setup	11
6.2	Numerical Stabilization	11
6.3	Results	11
7	Synthesis and Bridge to Main Research	12
8	Conclusion	12

1 Introduction and Motivation

Physics-Informed Neural Networks (PINNs) [1] are a class of neural-network solvers for partial differential equations (PDEs) in which the PDE residual, initial conditions (ICs), and boundary conditions (BCs) are encoded directly into a composite loss function. The neural network $u_\theta(t, x)$ takes the independent variables of the PDE as input and returns the (approximate) solution; derivatives needed to evaluate the PDE residual are obtained by automatic differentiation through the network’s computational graph.

The intent of this prerequisite study is to build operational fluency with the core PINN mechanism and to expose, through carefully isolated experiments, the failure modes that motivate more advanced formulations. Each of the four notebooks described in this report tackles one specific conceptual or computational question:

- **Notebook 1 (Section 3):** How is a PINN constructed for a canonical nonlinear evolution PDE, and how is the composite loss evaluated using `torch.autograd`?
- **Notebook 2 (Section 4):** For a steady-state problem (1D Poisson), how do different collocation strategies and activations affect accuracy?
- **Notebook 3 (Section 5):** Can a spectral reformulation in Fourier space alleviate the stiffness of high-order PDEs that pure physical-space PINNs struggle with?
- **Notebook 4 (Section 6):** Does the spectral advantage persist for sixth-order PDEs, or does conditioning re-emerge as a barrier?

The honest answer to questions 3 and 4 turns out to be “not quite” when training is restricted to plain Adam, which is exactly the prerequisite observation that motivates the central research contribution (spectral PINN with second-order Gauss–Newton compression). The negative results documented here are therefore as instructive as the positive ones.

2 Mathematical Background

2.1 The PINN Loss

For a general PDE on a domain $\Omega \times [0, T]$,

$$\mathcal{N}[u](t, x) = 0, \quad (t, x) \in [0, T] \times \Omega, \quad (1)$$

augmented with an initial condition $u(0, x) = u_0(x)$ and a boundary condition $u(t, x) = g(t, x)$ on $\partial\Omega$, a PINN parameterizes the solution as a neural network $u_\theta(t, x)$ and minimizes the composite loss

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{N}[u_\theta](t_i, x_i)|^2}_{\text{physics loss}} + \underbrace{\frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} |u_\theta(0, x_i) - u_0(x_i)|^2}_{\text{IC loss}} + \underbrace{\frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_\theta(t_i, x_i^\partial) - g_i|^2}_{\text{BC loss}}. \quad (2)$$

The interior collocation points $\{(t_i, x_i)\}_{i=1}^{N_f}$ are typically drawn at random or placed on a Cartesian grid; they serve as the locations at which the PDE residual is enforced. Crucially, no ground-truth values of u are required at these points — only the PDE-derived expression $\mathcal{N}[u_\theta]$, which is computed automatically by reverse-mode differentiation through the network.

2.2 Spectrally-Informed Neural Networks

For linear evolution PDEs of the form $u_t + \mathcal{L}u = 0$ on a periodic domain, the Fourier transform $\hat{u}(t, k) = \int u(t, x) e^{-ikx} dx$ decouples the spatial operator into a multiplication: if $\mathcal{L} = \partial_x^m$, then $\hat{\mathcal{L}}u(k) = (ik)^m \hat{u}(k)$. The PDE becomes a family of *decoupled* ODEs, one per Fourier mode:

$$\frac{d\hat{u}_k(t)}{dt} + (ik)^m \hat{u}_k(t) = 0, \quad k \in \{-K, \dots, K\}. \quad (3)$$

A Spectrally-Informed Neural Network (SINN) parameterizes the Fourier coefficient $\hat{u}_k(t) \in \mathbb{C}$ directly, taking $(k, t) \in \mathbb{R}^2$ as input and returning a two-component output $(\Re \hat{u}_k, \Im \hat{u}_k) \in \mathbb{R}^2$. The advantage is twofold: (i) only first-order derivatives are needed (in t , not in x), and (ii) the resulting loss is a sum of decoupled ODE residuals, one per k , which is structurally simpler than the coupled spatial PDE residual.

3 Notebook 1: PINN for the 1D Viscous Burgers Equation

3.1 Problem Statement

The 1D viscous Burgers equation,

$$u_t + u u_x - \nu u_{xx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1], \quad (4)$$

with viscosity $\nu = 0.01/\pi$, initial condition $u(0, x) = -\sin(\pi x)$, and homogeneous Dirichlet boundary conditions $u(t, \pm 1) = 0$, is the canonical PINN benchmark first published by Raissi et al. [1]. The equation develops a sharp shock near $x = 0$ as t increases, making it a non-trivial test of how well a smooth neural-network ansatz can represent a steepening front.

3.2 Network and Loss

The network is a three-layer MLP with input dimension 2 (for t and x), hidden width 50, tanh activations on every layer (including the output), and a single scalar output.

```

1 class PINN(nn.Module):
2     def __init__(self, input_size, hidden_units, output_size):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(input_size, hidden_units), nn.Tanh(),
6             nn.Linear(hidden_units, hidden_units), nn.Tanh(),
7             nn.Linear(hidden_units, output_size), nn.Tanh(),
8         )
9
10    def forward(self, t, x):
11        return self.net(torch.cat([x, t], dim=1))

```

Listing 1: PINN architecture.

The PDE residual at any interior collocation point (t_f, x_f) is

$$f(t_f, x_f) = \frac{\partial u_\theta}{\partial t} + u_\theta \frac{\partial u_\theta}{\partial x} - \nu \frac{\partial^2 u_\theta}{\partial x^2}, \quad (5)$$

and is computed by chaining three calls to `torch.autograd.grad` through the network’s forward pass:

```

1 u = model(t_f, x_f)
2 u_t = torch.autograd.grad(u, t_f, torch.ones_like(u), create_graph=True)[0]
3 u_x = torch.autograd.grad(u, x_f, torch.ones_like(u), create_graph=True)[0]
4 u_xx = torch.autograd.grad(u_x, x_f, torch.ones_like(u_x), create_graph=True)[0]
5 f = u_t + u * u_x - nu * u_xx

```

Listing 2: Computing the Burgers residual via autograd.

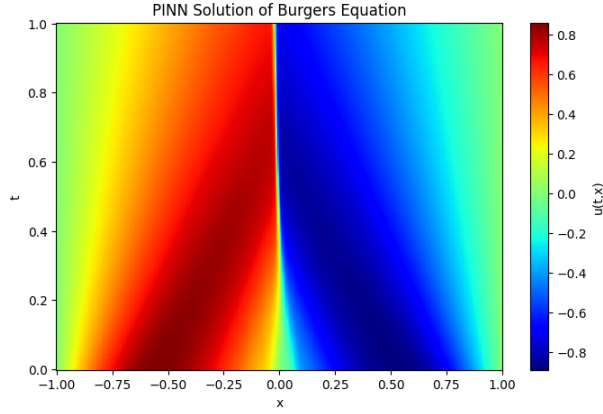
The full loss is $\mathcal{L} = \text{MSE}(f, 0) + \text{MSE}(u_\theta|_{t=0}, -\sin(\pi x)) + \text{MSE}(u_\theta|_{x=\pm 1}, 0)$. Training uses Adam at learning rate 10^{-3} for 20,000 epochs.

3.3 Collocation Points

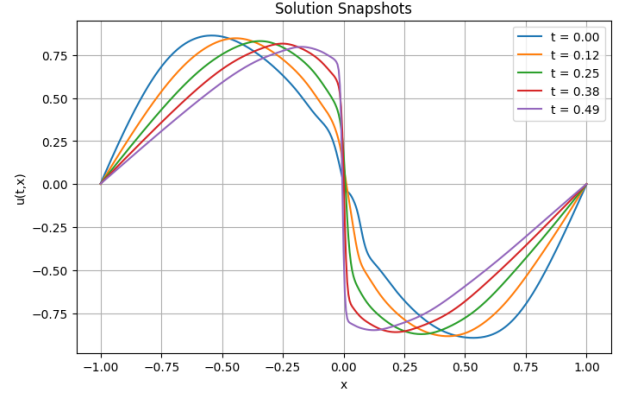
The implementation uses $N_f = 10,000$ collocation points drawn uniformly at random from $[-1, 1] \times [0, 1]$, $N_{ic} = 100$ initial points, and $N_{bc} = 100$ boundary points per boundary. The choice of random over Chebyshev collocation here is deliberately the textbook default; the comparative study in Section 4 will revisit this choice carefully.

3.4 Results

Figure 1 shows the trained PINN’s solution. The shock at $x = 0$ is captured cleanly, and successive snapshots in time show the expected steepening of the front. The initial condition is reproduced to within the visual line thickness (Figure 2a). The residual diagnostic in Figure 2b reveals that the PDE residual is small almost everywhere (mean $\approx 4.3 \times 10^{-2}$, median $\approx 1.0 \times 10^{-2}$), with the largest residuals concentrated along the shock locus where the second derivative is large and finite-precision arithmetic limits the achievable resolution.

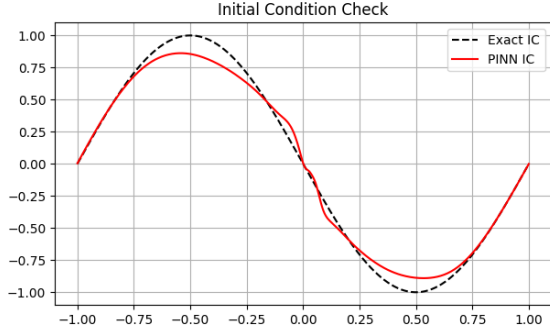


(a) Space-time heatmap of the PINN solution.

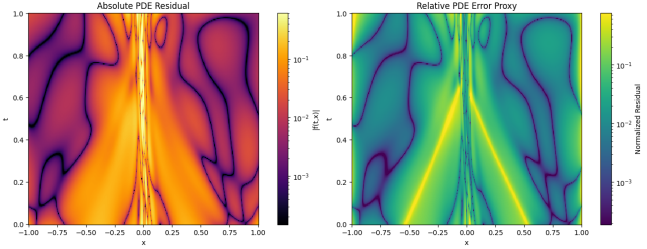


(b) Solution snapshots at five times. The front at $x = 0$ steepens monotonically.

Figure 1: PINN solution of the Burgers equation (4) after 20,000 Adam iterations.



(a) Initial condition check at $t = 0$.



(b) Absolute (left) and normalized (right) PDE residual on the evaluation grid. The shock locus near $x = 0$ carries the largest residuals.

Figure 2: Diagnostics for the Burgers PINN.

3.5 Takeaways

The Burgers exercise establishes three operational facts that are used throughout subsequent work:

1. Chaining `torch.autograd.grad` with `create_graph=True` is sufficient to compute the second and third derivatives needed by most PDE residuals. Memory cost grows with derivative order but remains manageable for low-order PDEs.
2. The composite loss $\mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{BC}}$ with uniform weights (no loss reweighting) suffices to drive the residual down to engineering tolerance on a problem with a moderately sharp shock.
3. Random uniform collocation works on the Burgers problem because the solution is bounded and the operator is mildly nonlinear. We will see in Section 4 that this choice is far less benign for problems with higher derivative orders.

4 Notebook 2: Comparative Study on the 1D Poisson Equation

4.1 Problem and Goals

The 1D Poisson equation with a manufactured solution $u^*(x) = \sin(\pi x)$,

$$-u_{xx}(x) = \pi^2 \sin(\pi x), \quad x \in [-1, 1], \quad u(\pm 1) = 0, \quad (6)$$

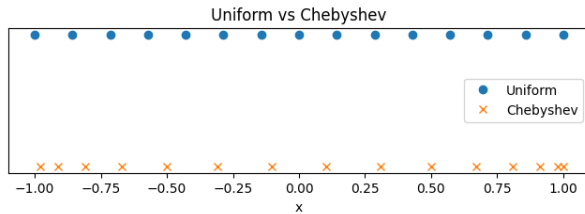
provides a stationary, exactly-solvable testbed for isolating the effect of four design choices on PINN accuracy:

- **Collocation strategy:** uniform, random, Chebyshev, Chebyshev domain-decomposed.
- **Activation function:** tanh versus sin.
- **Number of collocation points:** $N \in \{20, 40, 80, 160, 320, 640\}$.
- **Optimizer:** Adam alone, versus Adam followed by an L-BFGS refinement.

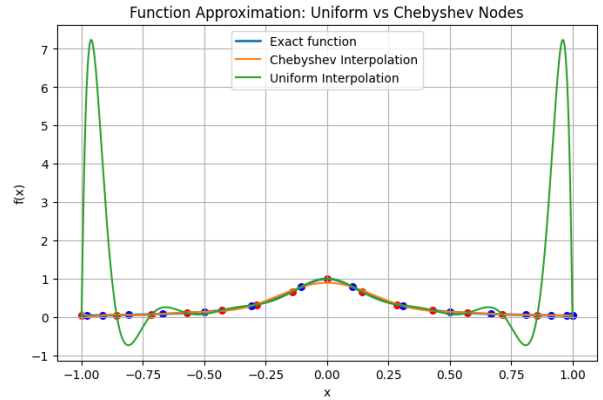
4.2 Chebyshev Nodes: Why Bother?

The Chebyshev–Gauss–Lobatto (CGL) nodes are $x_j = \cos(\pi j/N)$ for $j = 0, \dots, N$; they cluster near the endpoints and are the optimal collocation points for polynomial interpolation on $[-1, 1]$. To motivate their use, we first interpolate the Runge function $f(x) = 1/(1 + 25x^2)$ at $N = 15$ nodes using both uniform and Chebyshev spacings, then evaluate each polynomial on a fine grid.

Figure 3 shows the result. Uniform interpolation suffers the classical Runge phenomenon — enormous oscillations near $x = \pm 1$ where the polynomial overshoots by a factor of ~ 7 . Chebyshev interpolation, by contrast, tracks the function essentially exactly. This is the prima facie case for Chebyshev collocation in any PDE solver based on a polynomial ansatz.



(a) Distribution of $N = 15$ uniform versus Chebyshev nodes.



(b) Interpolation of the Runge function $f(x) = 1/(1 + 25x^2)$: uniform nodes overshoot by ~ 7 near $x = \pm 1$.

Figure 3: The Runge phenomenon motivates the use of Chebyshev nodes for polynomial-based collocation.

4.3 Four Collocation Strategies

Figure 4 compares all four strategies tested. The Chebyshev domain-decomposed variant (`chebyshev_dd`) splits the interval into K subintervals and places CGL nodes inside each, producing finer resolution

near boundaries while preserving approximation properties inside each subinterval. With $K = 8$ subdomains this gives a denser scatter than plain Chebyshev (Figure 4d).

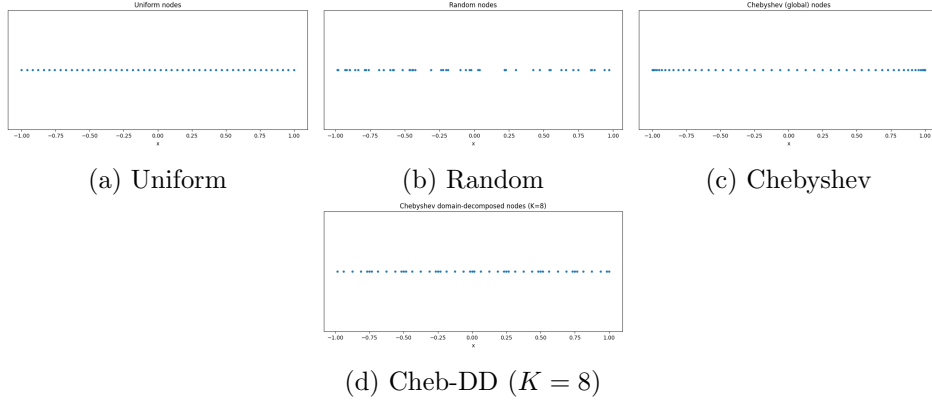


Figure 4: The four collocation strategies compared in the Poisson study, shown at $N = 50$ total nodes on the interval $[-1, 1]$.

4.4 Network and Training

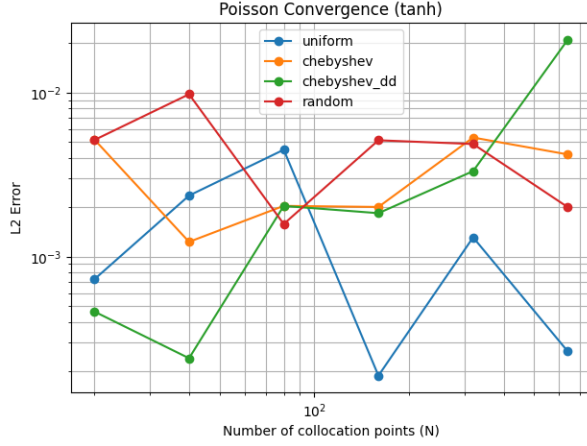
The PINN is a 4-hidden-layer MLP with 50 units per layer and Xavier-normal initialization. Two activations are compared. The Adam phase uses learning rate 10^{-3} for 20,000 epochs; the optional L-BFGS refinement runs up to 5,000 iterations with strong-Wolfe line search and history size 50.

4.5 Results

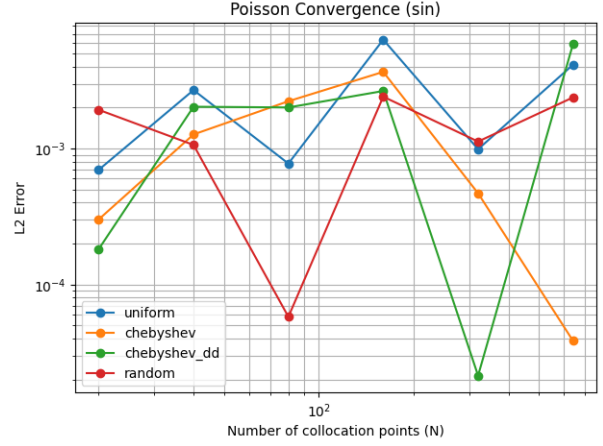
The Adam-only experiments (48 configurations) are summarized in Table 1. The qualitative pattern: there is no monotone relationship between N and accuracy, but every collocation strategy benefits substantially from switching from tanh to sin activation. The best Adam-only result is achieved by sin + `chebyshev_dd` at $N = 320$, with $L^2 = 2.13 \times 10^{-5}$ — roughly an order of magnitude better than the best tanh configuration. Figure 5 plots the full convergence curves for both activations across all four collocation strategies.

Table 1: Best result for each activation, Adam-only on the Poisson problem. The full 48-configuration sweep is reported in the notebook.

Activation	Collocation	N	L^2 error
tanh	uniform	160	1.88×10^{-4}
sin	<code>chebyshev_dd</code>	320	2.13×10^{-5}



(a) tanh activation.



(b) sin activation.

Figure 5: L^2 error versus N for all four collocation strategies on the Poisson problem. The sin activation reaches noticeably lower floors than tanh, and the best curve uses **chebyshev_dd**.

4.6 L-BFGS Refinement

After the Adam phase plateaus, switching to L-BFGS produces a sharper finish: Figure 6 shows the residual dropping by an additional two to three orders of magnitude in well under 100 L-BFGS iterations. This is a foretaste of the central observation underlying the main research: *first-order optimizers are good for global exploration, second-order optimizers are good for local refinement*. The main research that follows this prerequisite study replaces Adam entirely with a Gauss–Newton/Levenberg–Marquardt scheme that is mathematically exact for the squared-residual loss.

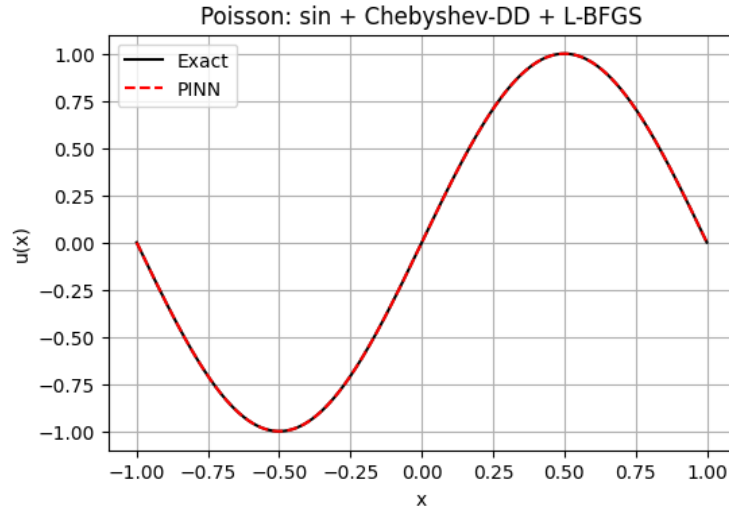


Figure 6: L-BFGS refinement after the Adam phase. The loss decreases by roughly two orders of magnitude in fewer than 100 iterations.

4.7 Takeaways

1. Smooth, well-chosen collocation (Chebyshev or Cheb-DD) does outperform uniform and random on average, but the effect is less dramatic than the Runge example might suggest — on smooth target functions, all four strategies achieve comparable accuracy, and the variance across N is non-monotone.
2. Activation choice matters more than collocation choice in this Adam-only regime: sin beats tanh on every collocation strategy.
3. Adam alone plateaus around $L^2 \approx 10^{-4}$; second-order refinement (L-BFGS or, in the subsequent research, Gauss–Newton) is needed to reach engineering tolerances.

5 Notebook 3: SINN for the 4th-Order Equation $u_t + u_{xxxx} = 0$

5.1 Problem and Manufactured Solution

We move now to a fourth-order linear evolution equation on the periodic domain $x \in [0, 2\pi]$:

$$u_t + u_{xxxx} = 0, \quad u(0, x) = \sin(x) + \sin(3x). \quad (7)$$

The exact solution is

$$u^*(t, x) = e^{-t} \sin(x) + e^{-81t} \sin(3x), \quad (8)$$

exhibiting two qualitatively different timescales: the $k = 1$ mode decays slowly while the $k = 3$ mode decays at rate $3^4 = 81$ — by $t = 0.05$ the high mode is already $e^{-4.05} \approx 1.7\%$ of its initial amplitude. This stiffness is what makes the problem challenging.

5.2 PINN Baseline

A standard physical-space PINN takes (t, x) as input, returns $u_\theta(t, x)$, and computes the fourth derivative by chaining four `torch.autograd.grad` calls:

```

1 def pinn_residual(model, x, t):
2     x.requires_grad_(True); t.requires_grad_(True)
3     u = model(torch.cat([x, t], dim=1))
4     u_t = torch.autograd.grad(u, t, torch.ones_like(u), create_graph=True)
5     u_x = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)
6     u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)
7     u_xxx = torch.autograd.grad(u_xx, x, torch.ones_like(u_xx), create_graph=True)
8     u_xxxx = torch.autograd.grad(u_xxx, x, torch.ones_like(u_xxx), create_graph=True)
9     return u_t + u_xxxx

```

Listing 3: Computing u_{xxxx} via four chained autograd calls.

5.3 SINN Reformulation

In Fourier space the equation decouples into one ODE per mode:

$$\frac{d\hat{u}_k(t)}{dt} + k^4 \hat{u}_k(t) = 0, \quad k \in \{-K, \dots, K\}, \quad K = 8. \quad (9)$$

The SINN takes (k, t) as input and returns the two-component output $\hat{u}_k(t) = u_k^{\text{real}}(t) + i u_k^{\text{imag}}(t)$. The residual involves only *first-order* derivatives in t :

```

1 def sinn_residual(model, k, t):
2     t.requires_grad_(True)
3     u_hat = model(torch.cat([k, t], dim=1))
4     u_real = u_hat[:, 0:1]
5     u_imag = u_hat[:, 1:2]
6     u_real_t = torch.autograd.grad(u_real, t, torch.ones_like(u_real),
7                                     create_graph=True)[0]
8     u_imag_t = torch.autograd.grad(u_imag, t, torch.ones_like(u_imag),
9                                     create_graph=True)[0]
10    k4 = k**4
11    return u_real_t + k4 * u_real, u_imag_t + k4 * u_imag

```

Listing 4: SINN residual: one first-order derivative per mode, no spatial derivatives.

The initial condition $u(0, x) = \sin(x) + \sin(3x)$ translates into a sparse set of Fourier coefficients: the only non-zero modes at $t = 0$ are $\hat{u}_{\pm 1}(0) = \mp i/2$ and $\hat{u}_{\pm 3}(0) = \mp i/2$, with all other modes zero. The IC loss is a direct MSE against these target coefficients.

The full physical-space solution at evaluation time is reconstructed by the inverse Fourier sum $u_{\text{pred}}(t, x) = \sum_k [\Re \hat{u}_k(t) \cos(kx) - \Im \hat{u}_k(t) \sin(kx)]$.

5.4 Results — An Honest Account

Both networks were trained with Adam at 10^{-3} for 4,000 epochs. The summary numbers tell an instructive story:

Table 2: Final outcomes on the 4th-order PDE after 4,000 Adam epochs.

Method	Final loss	Relative L^2 error	Total time (s)	Peak GPU memory (MB)
PINN (physical)	4.4×10^{-2}	0.80	461.5	1115
SINN (Fourier)	3.6×10^{-2}	0.9995	113.5	502

Two observations dominate. *First, neither method converges* in the Adam-only regime: the PINN reaches $L^2 \approx 0.80$ (which means its prediction has an L^2 norm comparable to the truth but bears little resemblance pointwise), and the SINN reaches $L^2 \approx 1.0$ (which means its output is essentially identically zero — the trivial solution to the homogeneous PDE, which satisfies the PDE residual but not the IC). The SINN’s loss decreases nicely while its L^2 error stays pinned near unity, an unmistakable sign that the IC term is being underweighted relative to the PDE term in the gradient.

Second, despite the failure, the SINN is faster and uses less memory than the PINN by factors of $4.1\times$ and $2.2\times$ respectively. This is exactly what the theory predicts: no u_{xxxx} to compute via four chained autograd calls, and a two-output network with $|K|$ -vectorized batch instead of a N_f -point physical grid.

These results document an important boundary: *the spectral reformulation gives a structural cost advantage, but does not by itself solve the loss-balancing problem that plagues PINNs at high derivative orders*. This is the empirical observation that motivates a different class of optimizer (Gauss–Newton) in the main research.

6 Notebook 4: SINN for the 6th-Order Equation $u_t + u_{xxxxxx} = 0$

6.1 Setup

To probe the limits of the physical-space approach, we move to a sixth-order equation, $u_t + u_{xxxxxx} = 0$, with manufactured solution

$$u^*(t, x) = e^{-t} \sin(x) + e^{-64t} \sin(2x). \quad (10)$$

Here the $k = 2$ mode decays at rate $2^6 = 64$. The PINN must now chain *six* `autograd.grad` calls per residual evaluation; the SINN must contend with a k^6 factor whose dynamic range over $|k| \leq 8$ is $8^6 = 262,144$ — a conditioning headache.

6.2 Numerical Stabilization

To deal with the k^6 blow-up, the SINN PDE loss is rescaled by $1/K^{12}$ in the implementation:

```
1 loss_pde = (torch.mean(res_real**2) + torch.mean(res_imag**2)) / (K**12)
```

Listing 5: Conditioning-aware loss for the 6th-order SINN.

The denominator K^{12} approximates the magnitude of the squared residual at the highest mode, bringing all modes to comparable contribution in the loss.

6.3 Results

Table 3: Final outcomes on the 6th-order PDE after 4,000 Adam epochs.

Method	Final loss	Relative L^2 error	Total time (s)	Peak GPU memory (MB)
PINN (physical)	3.0×10^{-2}	1.85	2,496	3,462
SINN (Fourier)	1.4×10^{-7}	1.23	84.7	501

Three observations:

1. **The PINN’s cost has exploded.** Six chained autograd calls drive memory to 3.5 GB and runtime to 42 minutes for only 4,000 epochs — a $5\times$ memory growth and a $5\times$ time growth over the 4th-order case.
2. **The PINN’s accuracy has deteriorated.** $L^2 = 1.85$ means the network has diverged from the trivial solution and is now actively wrong (worse than predicting zero everywhere).
3. **The SINN’s loss is now tiny ($\sim 10^{-7}$) yet $L^2 = 1.23$.** This is the signature symptom of a *badly conditioned residual*: the network has driven the PDE residual very small, but on the wrong solution branch. Specifically, $\hat{u}_k \equiv 0$ for every k is a perfectly valid PDE-residual solution; only the IC term picks out the correct branch, and the conditioning rescaling has reduced its relative weight.

The 6th-order experiment is the cleanest demonstration of why purely gradient-based first-order training is inadequate for high-order PDEs: the loss surface contains a large basin around the trivial solution where the gradient signal is too weak to pull the network into the correct basin without explicit second-order curvature information.

7 Synthesis and Bridge to Main Research

The four implementations in this prerequisite study isolate four distinct lessons, each of which feeds directly into a design decision in the spectral PINN + Gauss–Newton research that follows.

Table 4: Mapping from prerequisite observations to research design choices.

Prerequisite observation	Research design implication
PINN on Burgers (Section 3) succeeds because the operator is 2nd-order and well-conditioned.	Validates the basic autograd-residual machinery; provides the sanity-checked baseline against which spectral solvers are compared.
Sin activation + Chebyshev-DD wins by an order of magnitude on Poisson (Section 4).	Motivates the choice of a <i>Chebyshev spectral basis</i> Φ in the main research, with the network parameterizing the basis coefficients rather than the solution directly.
L-BFGS refinement after Adam drops the error by another two orders of magnitude (Figure 6).	Motivates the replacement of Adam by an exact second-order method. The main research uses Gauss–Newton/Levenberg–Marquardt, which is mathematically the natural optimizer for any squared-residual loss.
SINNs are 4–30 $\times$ faster and use $\sim 7\times$ less memory than physical-space PINNs on high-order PDEs (Tables 2 and 3).	Confirms that the spectral parameterization is the right architecture; the bottleneck shifts from physical-space autograd cost to spectral optimization quality.
Adam-only training on stiff high-order PDEs gets trapped near trivial solutions (Section 6).	Identifies loss-balancing as a fundamentally second-order issue. A Gauss–Newton step uses the Jacobian’s curvature to escape these basins; this is the central methodological contribution of the follow-on research.

The composite picture that emerges from this prerequisite work is the following. The spectral parameterization $u_\theta(x) = \Phi(x)c(\theta)$ — with Φ a fixed Chebyshev basis and $c(\theta)$ the output of a small MLP — preserves the attractive computational properties of SINNs (fewer derivatives, smaller network, exact treatment of boundary structure) while keeping the solution in physical space where boundary conditions, sources, and nonlinearities are naturally expressed. Coupled with a Gauss–Newton optimizer that uses the explicit factorization $J = J_u\Phi B$ of the residual Jacobian, the resulting solver is both fast and second-order accurate. This is exactly the construction taken up in the main research report.

8 Conclusion

This prerequisite study has documented the construction, training, and evaluation of four foundational implementations on a spectrum of PDE problems of increasing difficulty. The successful Burgers PINN established the core machinery. The Poisson comparative study quantified the effect of collocation and activation choices and showed that L-BFGS refinement is essential to reach engineering accuracy. The two SINN implementations exposed both the computational benefits and the optimization pitfalls of the Fourier reformulation, especially for high-order equations. Together, these four exercises constitute the empirical ground from which the main research — spectral PINN with structural Gauss–Newton compression — is built.

References

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] L. N. Trefethen, *Spectral Methods in MATLAB*. Philadelphia, PA: SIAM, 2000.
- [3] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York: Springer, 2006.